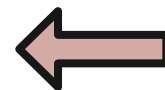
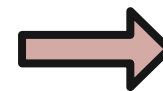
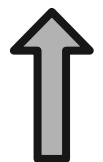


Introduction to iOS Development



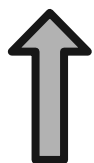


Hello, world! I'm Kamryn.



I'm passionate about mobile development & began coding in Swift & with Xcode in 2017.

I took CS50 last year :)

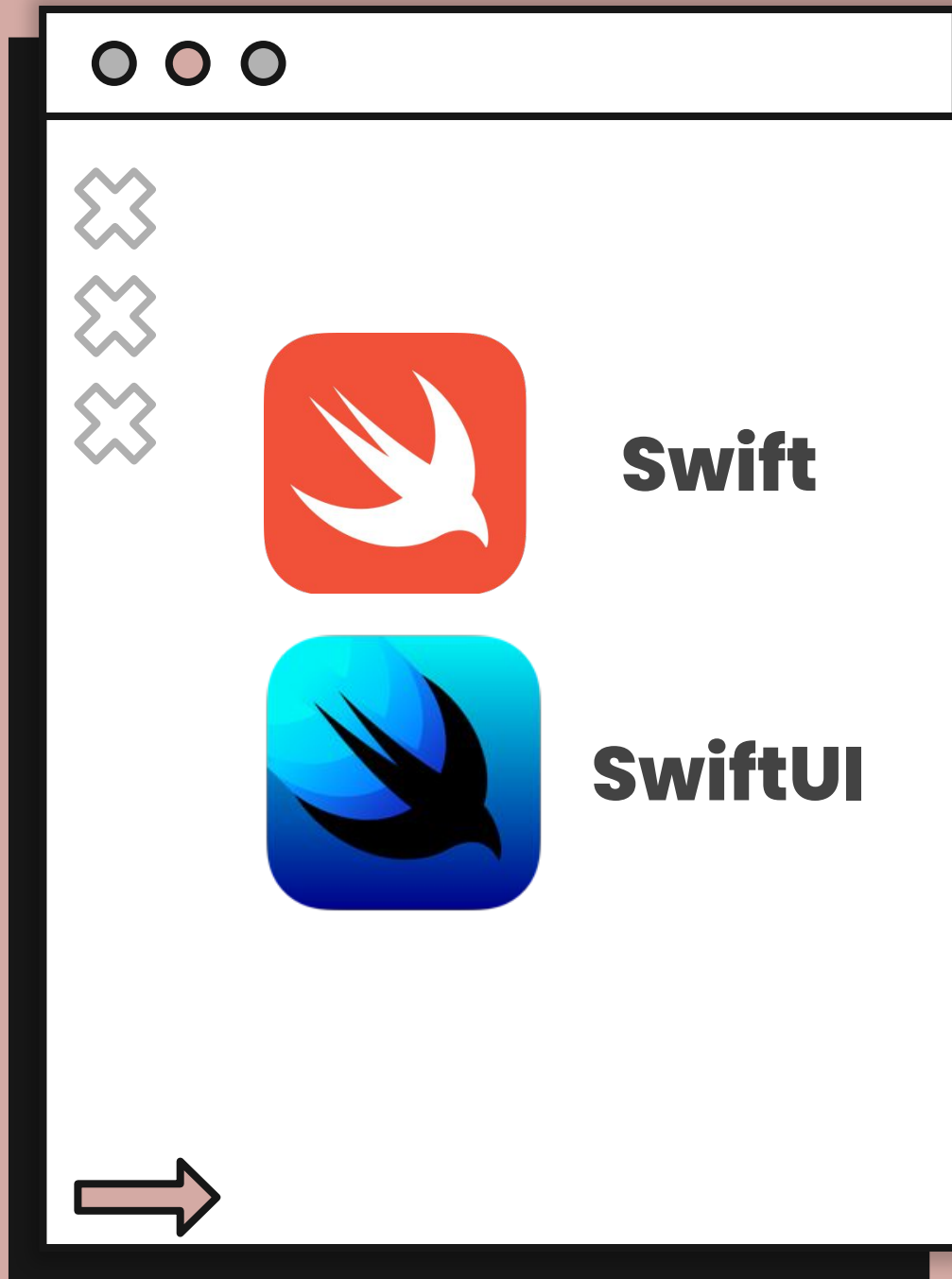




Today's Seminar

- What is **iOS Development**?
- How do we code in **Swift**? How do we use **Xcode**?
- Let's create a project ✨
- Where can we go from here?
- Additional Resources





What is iOS development?

Creating applications designed for Apple's framework.

Tools required: Access to Apple device, Xcode.



The Swift Programming Language



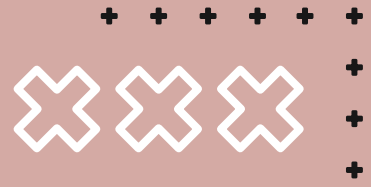
```
4  
5  
6  
7  
8  
9 print("hello, world")  
10  
11  
12  
13  
14  
15  
16  
17  
18  
19  
20  
21
```



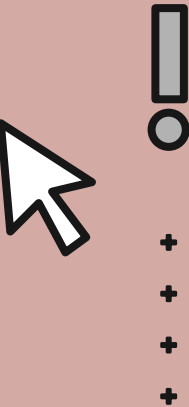
Python vs. Swift

Variables in Python

```
1
2 # Variables in Python
3 variable = "Hello, world"
4 variable = "Hello, Swift"
5 print(variable)
6 |
```

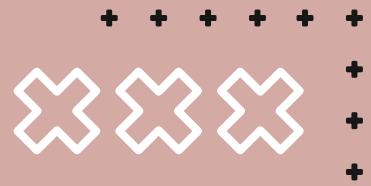


```
10 // Creating a variable in Swift
11 var greeting = "Hello, Python!"
12 print(greeting)
13
14 var psetsLeft = 0
15 print(psetsLeft)
16
17 // Updating variables
18 greeting = "Hello, Swift!"
19 print(greeting)
```



Python vs. Swift

Variables



```
# Variables in Python
variable = "Hello, world"
variable = "Hello, Swift"
print(variable)
```

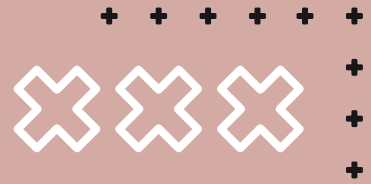
```
// Creating a variable in Swift
var greeting = "Hello, Python!"
print(greeting)
```

```
var psetsLeft = 0
print(psetsLeft)
```

```
// Updating variables
greeting = "Hello, Swift!"
print(greeting)
```

Python vs. Swift

Arrays & Dictionaries in Python



```
# Arrays & Dictionaries
```

```
array_of_courses = ["CS50", "CS51", "CS121", "CS124"]
```

```
dictionary_of_course = {
```

```
    "name": "CS50",
```

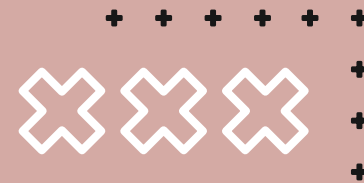
```
    "description": "Introduction to Computer Science",
```

```
    "year": 2022
```

```
}
```

Python vs. Swift

Arrays & Dictionaries in Swift



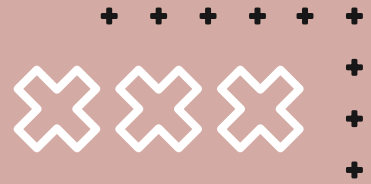
```
// Arrays & Dictionaries
```

```
var arrayOfCourses = ["CS50", "CS51", "CS121", "CS124"]
```

```
var dictionaryOfCourse : [String : Any] = [  
    "name": "CS50",  
    "description": "Introduction to Computer Science",  
    "year": 2022  
]
```

Python vs. Swift

Loops in Python



```
# Loops in Python
```

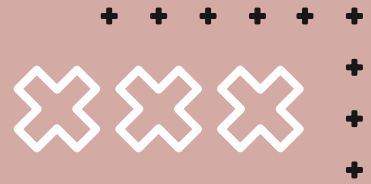
```
playlist = ["You Belong With Me", "Drivers License", "Decode"]  
for song in playlist:  
    print("I'm listening to", song)
```

```
# Loop through & print every other song.
```

```
for i in range(0, len(playlist)):  
    if (i % 2) == 0:  
        print(playlist[i])
```

Python vs. Swift

Loops in Swift

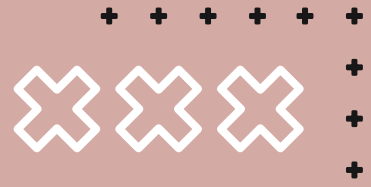


```
// Loops in Swift
var playlist = ["You Belong With Me", "Drivers License", "Decode"]
for song in playlist
{
    print("I'm listening to", song)
}

// Loop through & print every other song.
for i in 0..
```

Python vs. Swift

Functions in Python

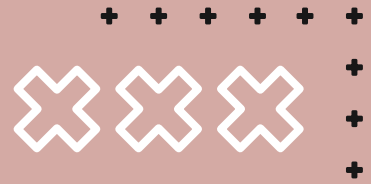


```
# Functions in Python
def say_hello(name):
    return "hello," + name

print(say_hello("world"))
```

Python vs. Swift

Functions in Swift



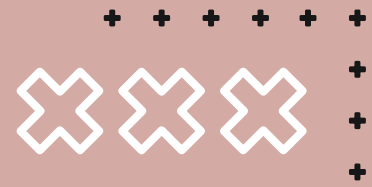
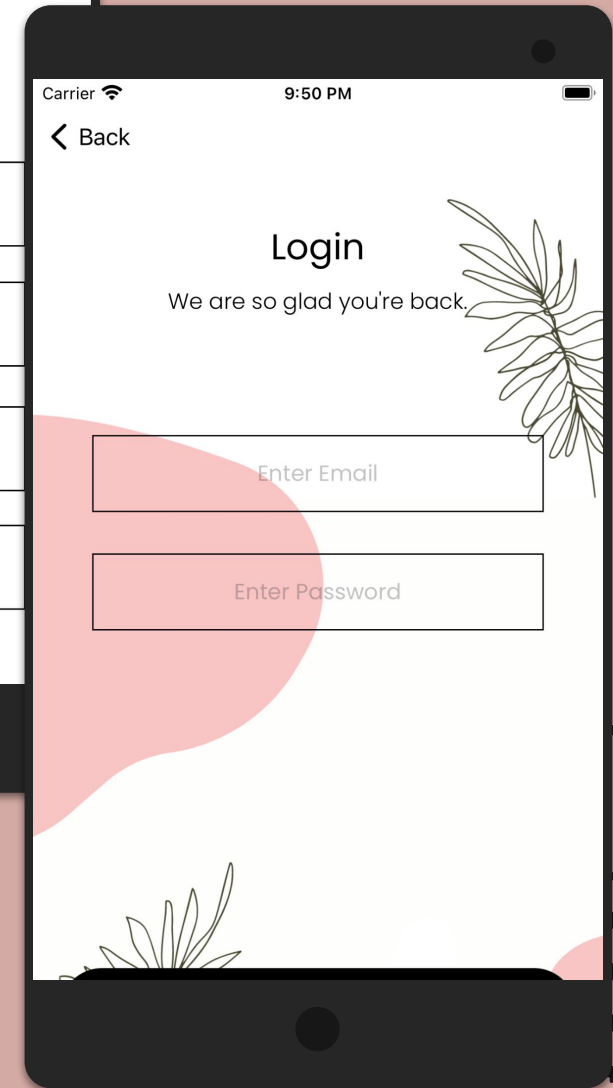
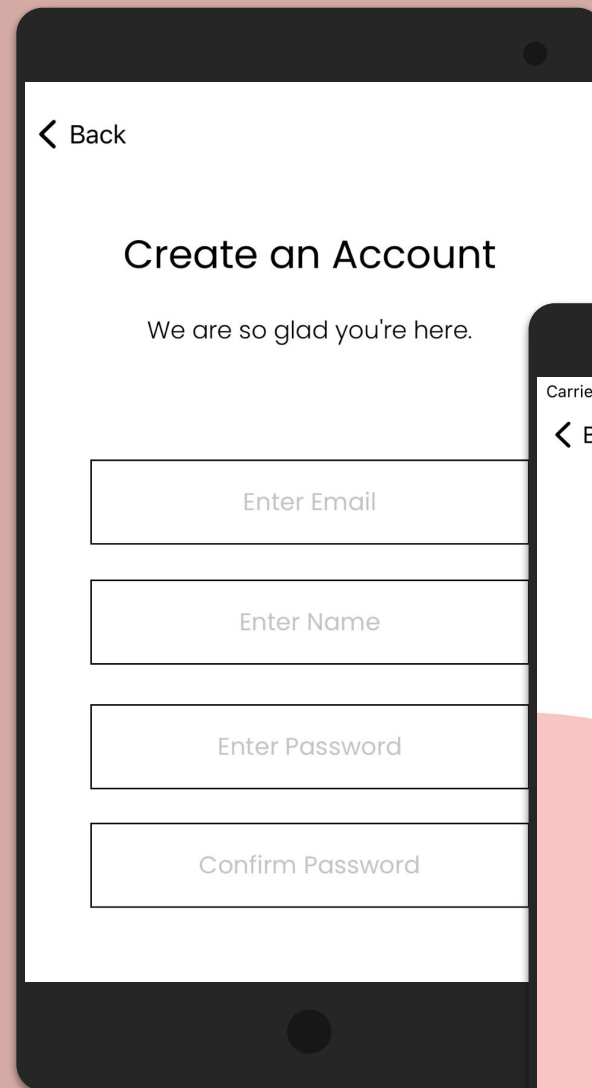
```
// Functions in Swift
func say_hello(name : String) -> String
{
    return "Hello, \(name)"
}

print(say_hello(name: "CS50"))
```

A Main Component

Classes & Objects in Swift

Think of each page of an app like a separate object. Today's app will work by defining a class (+ a new code file) for each screen, which we call "view controllers."

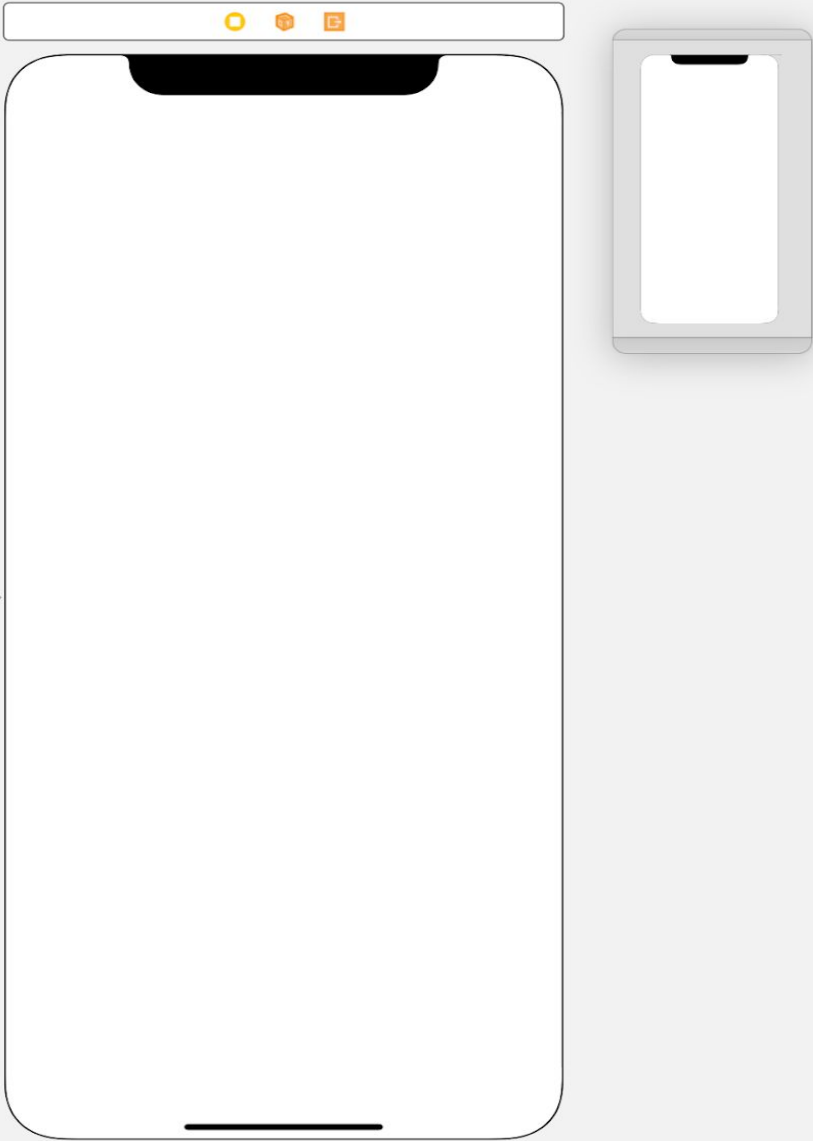




Let's get started with Xcode.

- ▼ NameOfYourApp
 - ▼ NameOfYourApp
 - AppDelegate
 - SceneDelegate
 - ViewController
 - Main
 - Assets
 - LaunchScreen
 - Info
 - NameOfYourApp
 - ▼ NameOfYourAppTests
 - NameOfYourAppTests
 - ▼ NameOfYourAppUITests
 - NameOfYourAppUITests
 - NameOfYour...tsLaunchTests

- NameOfYourApp > NameOfYourApp > Main > Main (Base) > View Controller Scene > View Controller > View
 - ▼ View Controller Scene
 - ▼ View Controller
 - View
 - Safe Area
 - First Responder
 - Exit
 - Storyboard Entry Point



View

Content Mode: Scale To Fill

Semantic: Unspecified

Tag: 0

Interaction: ☒ User Interaction Enabled
☐ Multiple Touch

Alpha: 1

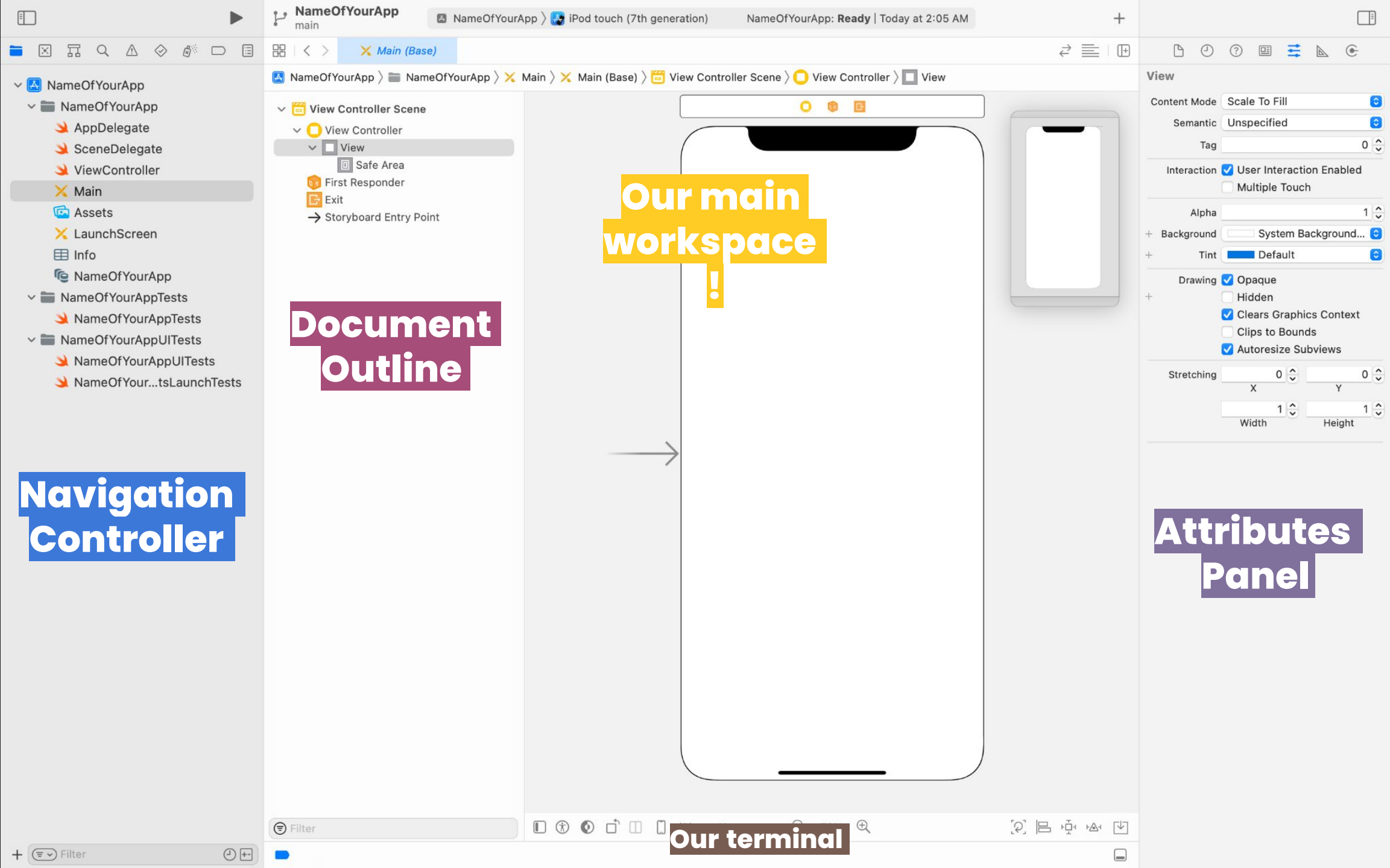
+ Background: System Background...

+ Tint: Default

Drawing: ☒ Opaque
☐ Hidden
☒ Clears Graphics Context
☐ Clips to Bounds
☒ Autorelease Subviews

Stretching: X: 0 Y: 0
Width: 1 Height: 1





Document Outline

Navigation Controller

Our main workspace

Attributes Panel

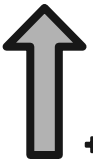
Our terminal



Let's create a project.



We will be doing a live demo. However, the essential instructions will also be in the following slides with screenshots.





Project Demonstration Instructions





Create your iOS project:

- 1) Open Xcode and select “Create a new Xcode project”
- 2) Select iOS → App
- 3) Give your app a name (the bundle identifier can be your website reversed, i.e. com.YourName)
- 4) Select Interface: Storyboard.
- 5) Select Language: Swift.
- 6) Save to a location on your computer (like Desktop or Documents)
- 7) Click create!

If you can't find your Main.storyboard, then you may have forgotten to select the interface as Storyboard !?



NameOfYourApp main NameOfYourApp > iPod touch (7th generation) NameOfYourApp: Ready | Today at 2:03 AM

NameOfYourApp

- NameOfYourApp
 - AppDelegate
 - SceneDelegate
 - ViewController
 - Main
 - Assets
 - LaunchScreen
 - Info
 - NameOfYourApp
- NameOfYourAppTests
 - NameOfYourAppTests
- NameOfYourAppUITests
 - NameOfYourAppUITests
 - NameOfYour...tsLaunchTests

General Signing & Capabilities Resource Tags Info Build Settings Build Phases Build Rules

PROJECT

- NameOfYourApp

TARGETS

- NameOfYourApp
- NameOfYourAppTests
- NameOfYourAppUIT...

Identity

Display Name: NameOfYourApp

Bundle Identifier: com.KamrynOhly.NameOfYourApp

Version: 1.0

Build: 1

Deployment Info

iOS 15.5

- ☒ iPhone
- ☒ iPad
- ☐ Mac Catalyst

Main Interface: Main

Device Orientation

- ☐ Portrait
- ☐ Upside Down
- ☐ Landscape Left
- ☐ Landscape Right

Status Bar Style: Default

- ☐ Hide status bar
- ☐ Requires full screen
- ☐ Supports multiple windows

App Icons and Launch Images

App Icons Source: AppIcon

- ☐ Include all app icon assets

Identity and Type

Name: NameOfYourApp

Location: Absolute

Containing directory

Full Path: /Users/kamrynohly/Desktop/
NameOfYourApp/
NameOfYourApp.xcodeproj

Project Document

Project Format: Xcode 13.0-compatible

Organization

Class Prefix

Text Settings

Indent Using: Spaces

Widths: 4 4

Tab Indent

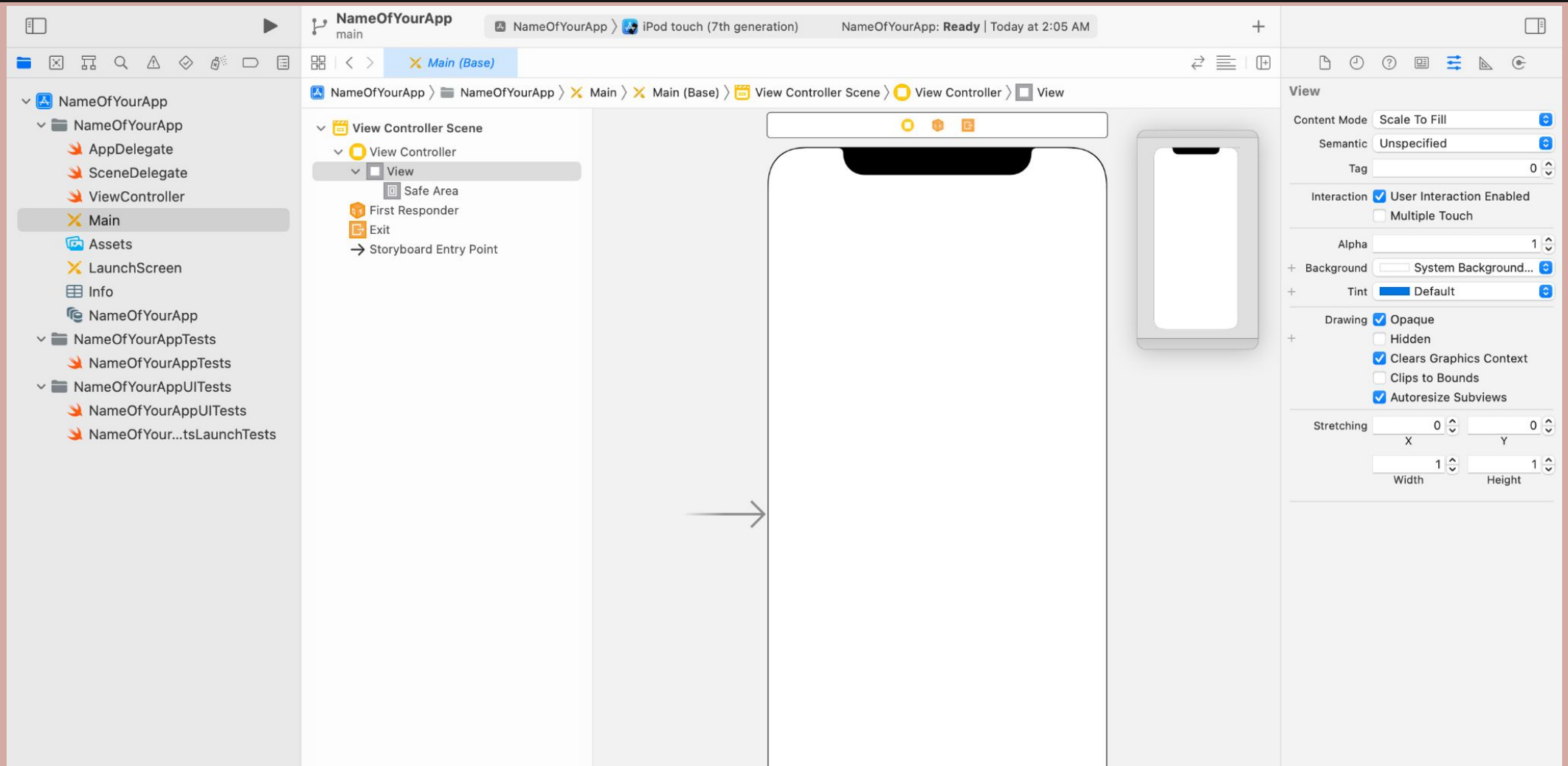
☒ Wrap lines

Once you've created your app, you should open up automatically to a page like this one! Let's click on the "Navigation Panel" on the far left and select "Main." The name may be slightly different, like "Main.storyboard"



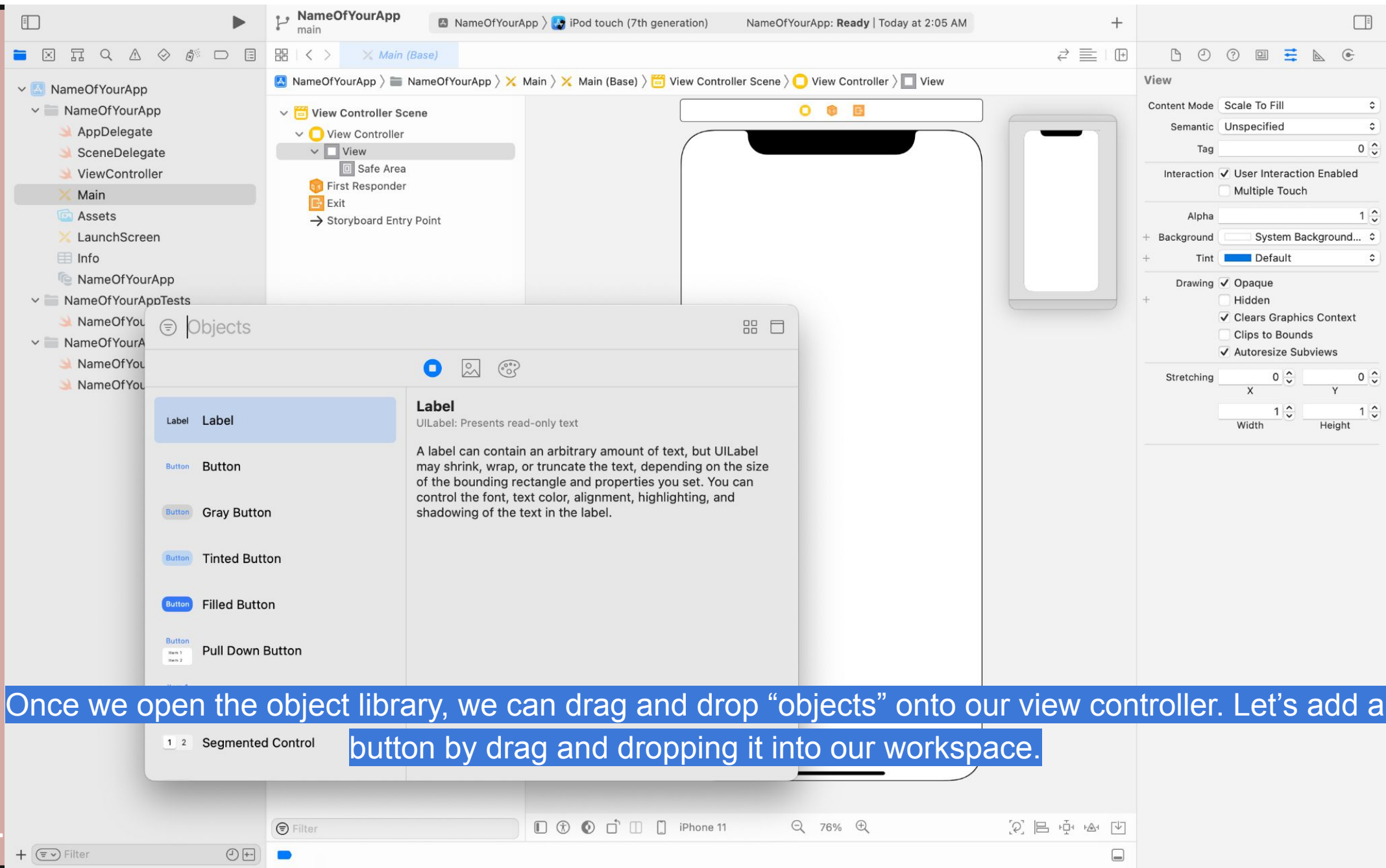


here ↓



Now, we should have our “View Controller” or the little iPhone, which we can work with more directly!
Let’s start by adding a button onto our screen. We can do this by opening the Object Library with the little plus button on the top-right of our screen.





here ↓

The screenshot shows the Xcode IDE with a storyboard open. The storyboard contains a single view with a blue button labeled "I'm a button!". The Identity inspector panel on the right is open, showing the class "ViewController" and the module "NameOfYourApp". The "Storyboard ID" field is empty. The "Document" section shows the "Label" as "Xcode Specific Label" and the "Object ID" as "BYZ-38-t0r".

Now that we have a button added, we want to be able to execute code when the button is clicked. Let's first check what code file our view controller is connected to by clicking on the identity panel (the little ID in the attributes panel), and the Class defined ViewController is the name of our code file.



here ↓

here →

UI Inspector menu options:

- Show Editor Only
- Preview
- Assistant
- Layout
- ✓ Constraints
 - Intrinsic Size Constraints Contributing To Ambiguity
 - Involved Views For Selected Constraints
 - Layout Rectangles
 - Bounds Rectangles
- ✓ Device Bezels
- ✓ Placeholder Backgrounds
- ✓ Document Outline
- ✓ Minimap
- ✓ Snap to Guides
- ✓ Automatically Refresh Designable Views

Document properties:

- Label: Xcode Specific Label
- Object ID: BYZ-38-t0r
- Lock: Inherited - (Nothing)
- Localizer Hint: Comment For Localizer

We can open our code side-by-side with our view controller by selecting the paragraph button and selecting “Assistant.” If we then click in the middle of our view controller, it should automatically load the correctly corresponding file.











NameOfYourApp main | iPod touch (7th generation) | NameOfYourApp: Ready | Today at 2:08 AM | 1

Main (Base)

View Controller Scene

- View Controller
 - View
 - Safe Area
 - I'm a button!
 - Label
 - First Responder
 - Exit
 - Storyboard Entry Point

Label

Text: Plain
Label
Color: Default (Label Color)
Font: System 17.0
Dynamic Type: Automatically Adjusts Font
Alignment: Left
Lines: 1
Behavior: ☒ Enabled
Highlighted
Tooltip: Shows Expansion Text
Baseline: Align Baselines
Line Break: Truncate Tail
Autoshrink: Fixed Font Size
Tighten Letter Spacing
Highlighted: Default
Shadow: Default
Shadow Offset: 0 Width, -1 Height
View
Content Mode: Left
Semantic: Unspecified
Tag: 0
Interaction: ☐ User Interaction Enabled
☐ Multiple Touch
Alpha: 1
Background: Default
Tint: Default
Drawing: ☐ Opaque
☐ Clips to Bounds
☒ Autocomplete Subviews
Stretching: 0 X, 0 Y
Width: 1, Height: 1
Installed

```
1 //
2 // ViewController.swift
3 // NameOfYourApp
4 //
5 // Created by Kamryn Ohly on 11/16/22.
6 //
7
8 import UIKit
9
10 class ViewController: UIViewController {
11
12     override func viewDidLoad() {
13         super.viewDidLoad()
14         // Do any additional setup after loading
15         // the view.
16     }
17
18     @IBAction func buttonClicked(_ sender:
19         UIButton) {
20         // This is a function that is called by a
21         // user tapping on the button on our
22         // screen / view controller!
23         // Let's do something. Let's add a label,
24         // then make the label change when the
25         // button is tapped!
```

After adding our labels, we can also use the attributes panel on the far right to decorate them / improve the UI design.





The screenshot displays the Xcode development environment for an app named "NameOfYourApp". The interface is divided into three main sections:

- Left Panel (Storyboard):** Shows a "View Controller Scene" with a "View Controller" containing a "View". The "View" has a "Safe Area" and a "Click me!" button. A "Button Response" is connected to the button. The "First Responder" is set to "Exit".
- Center Panel (Swift File):** Shows the "ViewController.swift" file. The code includes a comment about connecting an OUTLET, an @IBOutlet declaration for "buttonResponse", and an @IBAction for "buttonClicked".
- Right Panel (Label Inspector):** Shows the "Label" inspector with various properties like Text, Color, Font, Dynamic Type, Alignment, Lines, Behavior, Tooltip, Baseline, Line Break, Autoshrink, Highlighted, Shadow, Shadow Offset, Content Mode, Semantic, Tag, Interaction, Alpha, Background, and Tint.

Next, we want to link the label that we want to update as an OUTLET in our code. An outlet is a connection where we are accessing information or updating it, unlike an action which requires user interaction. We can do this by clicking on the label we want the button to change, and we **CONTROL +** drag the connection above our `viewDidLoad()` function as an OUTLET.





The way we can update labels in Swift is by accessing the name of our outlet buttonResponse's text attribute and updating it like a typical variable.

The screenshot displays the Xcode development environment. On the left, a storyboard for an iPhone 11 shows a pink view controller with a white header bar containing three colored circles (yellow, orange, red). The main area of the view contains the text "Click the button if you're having a great day!", a small laptop icon, and a blue button labeled "Click me!". A grey arrow points from the text to the button. On the right, the "ViewController.swift" file is open, showing Swift code. The code includes a comment about the file's creation and an IBOutlet for a UILabel named "buttonResponse". The viewDidLoad method calls super.viewDidLoad(). The buttonClicked method is connected to the "Click me!" button in the storyboard. Inside buttonClicked, the text of "buttonResponse" is updated to "yay!! 🥳".

```
2 // ViewController.swift
3 // NameOfYourApp
4 //
5 // Created by Kamryn Ohly on 11/16/22.
6 //
7
8 import UIKit
9
10 class ViewController: UIViewController {
11
12     // Connect an OUTLET (or a piece of information that you want
13     // to access or update) by control dragging above the
14     // viewDidLoad function.
15     @IBOutlet weak var buttonResponse: UILabel!
16
17     override func viewDidLoad() {
18         super.viewDidLoad()
19         // Do any additional setup after loading the view.
20     }
21
22     // Our button is called an ACTION, as it is a piece of our
23     // user interface that is interacted with by the user.
24     @IBAction func buttonClicked(_ sender: UIButton) {
25         // This is a function that is called by a user tapping on
26         // the button on our screen / view controller!
27         // Let's do something. Let's add a few labels, then make
28         // a label change when the button is tapped!
29
30         // We can change the text by using our variable name of
31         // our label and accessing the .text attribute.
32         buttonResponse.text = "yay!! 🥳"
33     }
34 }
```





The screenshot shows the Xcode development environment. On the left, a pink simulator displays a button labeled "Click me!". In the center, a "Mac" menu is open, showing a list of iOS simulators. The "iPhone 11" is highlighted. On the right, the "ViewController.swift" file is open, showing Swift code. The code includes a class `ViewController` that inherits from `UIViewController`. It has an `@IBOutlet` for a button and an `@IBAction` for a button click event. The `viewDidLoad` method is overridden, and the `buttonClicked` method is implemented to change the text of the button.

```
// ViewController.swift
// NameOfYourApp
// Created by Kamryn Ohly on 11/16/22.
//

import UIKit

class ViewController: UIViewController {

    // Connect an OUTLET (or a piece of information that you want
    // to access or update) by control dragging above the
    // viewDidLoad function.
    @IBOutlet weak var buttonResponse: UILabel!

    override func viewDidLoad() {
        super.viewDidLoad()
        // Do any additional setup after loading the view.
    }

    // Our button is called an ACTION, as it is a piece of our
    // user interface that is interacted with by the user.
    @IBAction func buttonClicked(_ sender: UIButton) {
        // This is a function that is called by a user tapping on
        // the button on our screen / view controller!
        // Let's do something. Let's add a few labels, then make
        // a label change when the button is tapped!

        // We can change the test by using our variable name of
        // our label and accessing the .text attribute

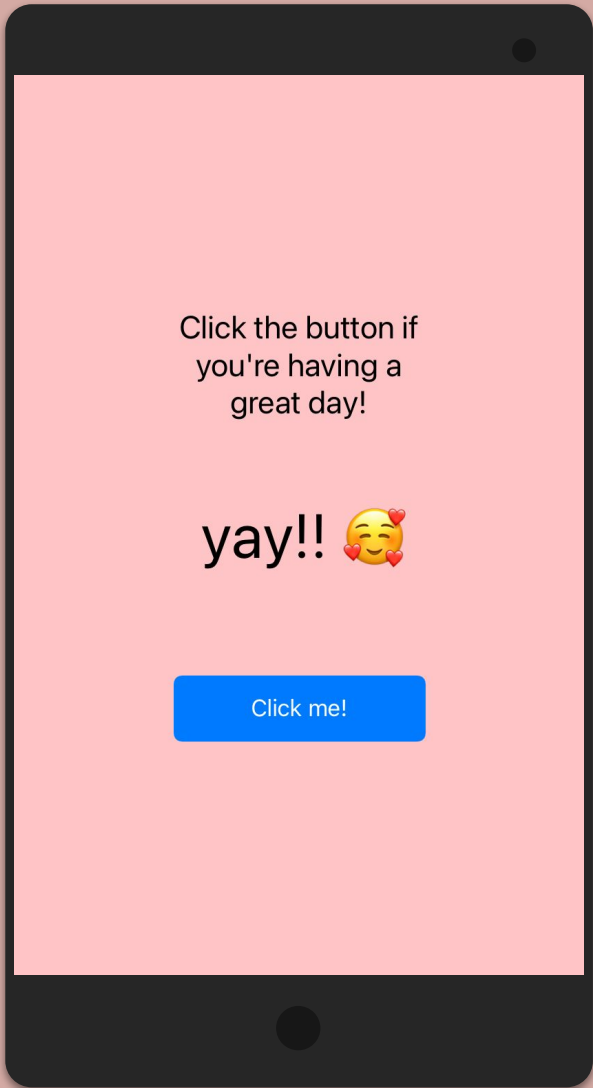
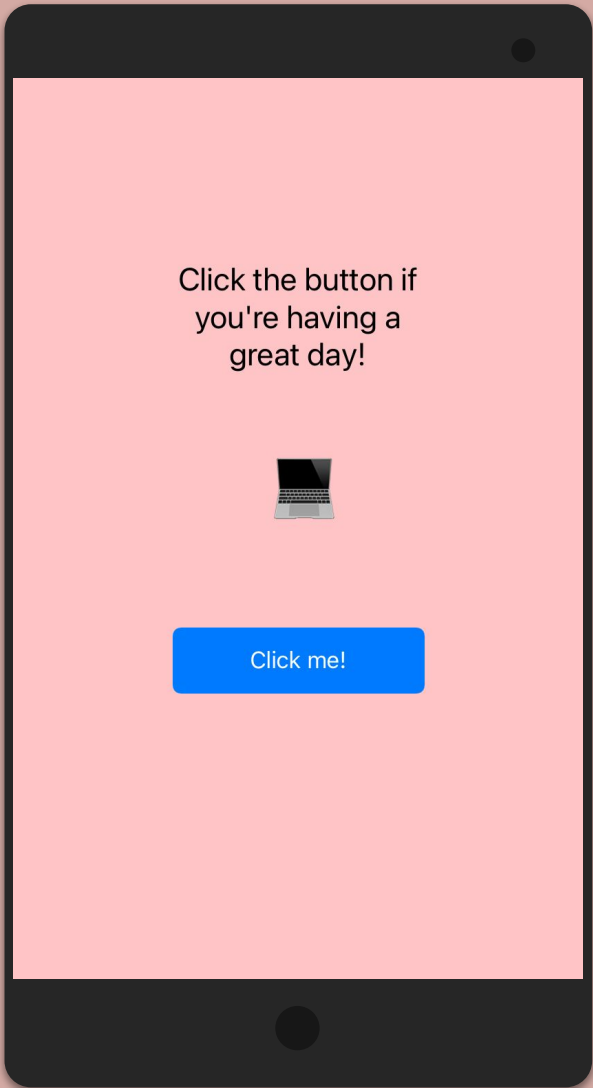
    }
}
```

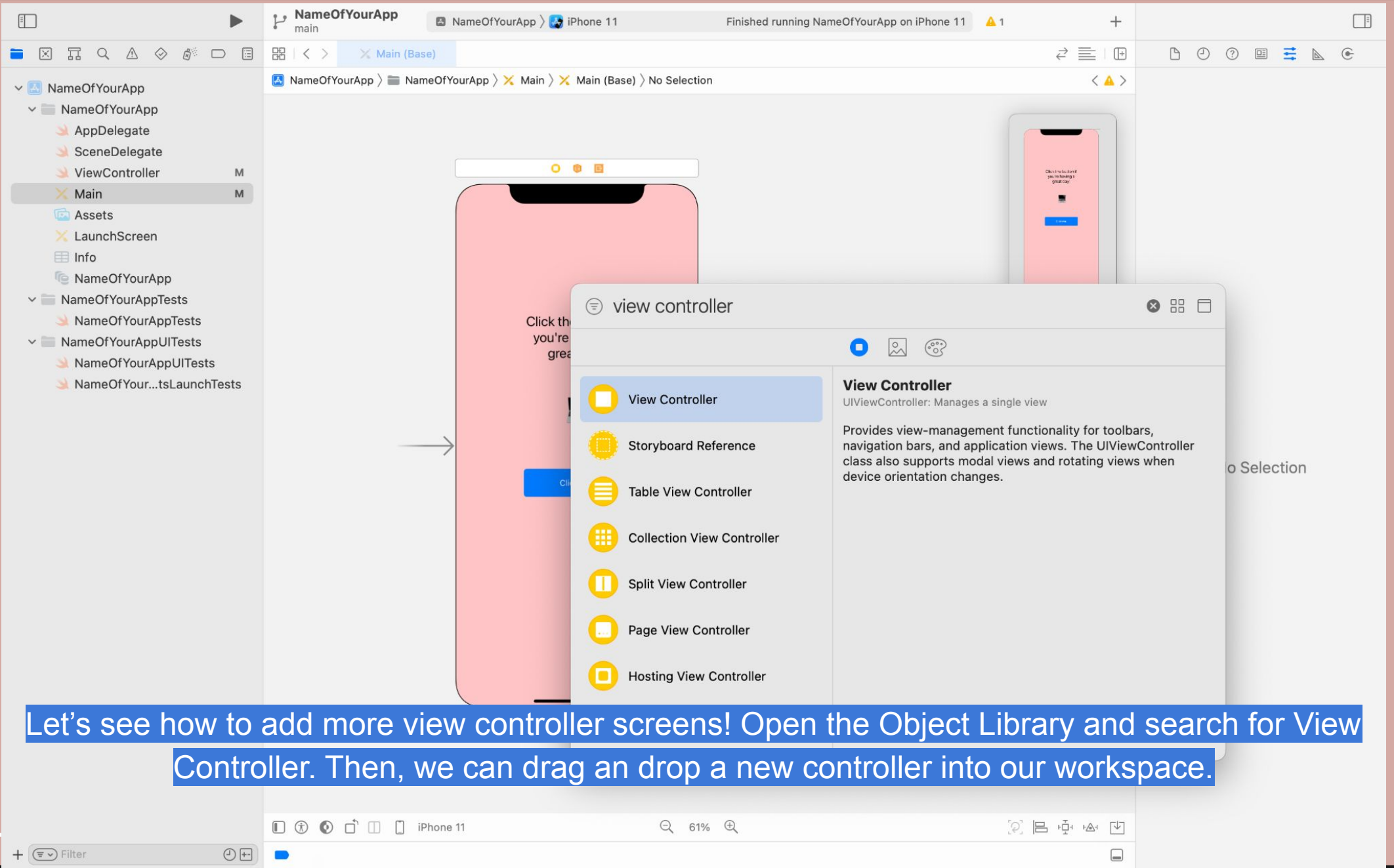
To run our code, we first need to make sure that the size of our view controller matches the simulator. Confirm that the iPhone 11 in the bottom left matches the top bar to the right of the Play button.





Our simulation. The simulator can take 2-5 minutes to initially load!

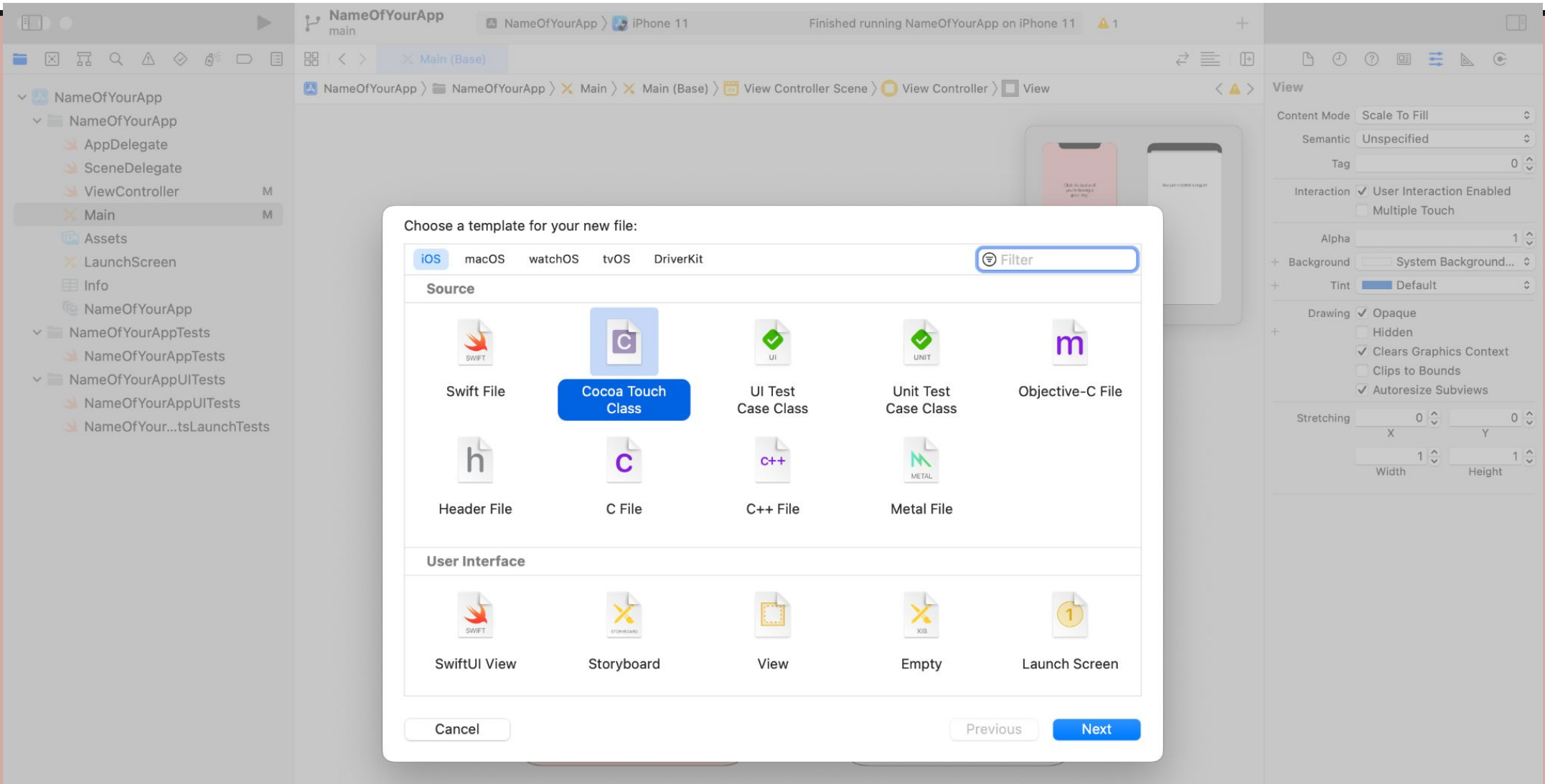




Let's see how to add more view controller screens! Open the Object Library and search for View Controller. Then, we can drag and drop a new controller into our workspace.

The screenshot displays the Xcode IDE with a storyboard for an app named "NameOfYourApp". The storyboard contains two view controller scenes. The first scene, labeled "Main", has a pink background and contains a button labeled "Click me!". The second scene, labeled "View Controller", has a white background and contains a button labeled "go to next page". A small inset window shows a preview of the "go to next page" button. The right-hand pane shows the "Button" properties for the selected button, including settings for Type, Style, Title, Subtitle, Alignment, Foreground, Background, Image, Background Configuration, Content Insets, Drawing, Role, Menu, Pointer, Accessibility, Shadow Offset, Width, Height, Reverses On Highlight, Shows Touch On Highlight, Highlighted Adjusts Image, Disabled Adjusts Image, Line Break, Drag and Drop, and Behavior. The "Main" view controller is selected in the left-hand pane.

We want to connect our first screen to the next. Let's add another button and create a SEGUE, or a transition to the next screen. We can do this by tapping on the button, holding CONTROL, and dragging to the middle of our next view controller screen. Choose any transition, like "Show."



Now, we must make a new Swift file to correspond to our new View Controller. We can do this by using File -> New File. It is easiest to use a Cocoa Touch Class, as this will give us some predefined code.



Choose options for your new file:

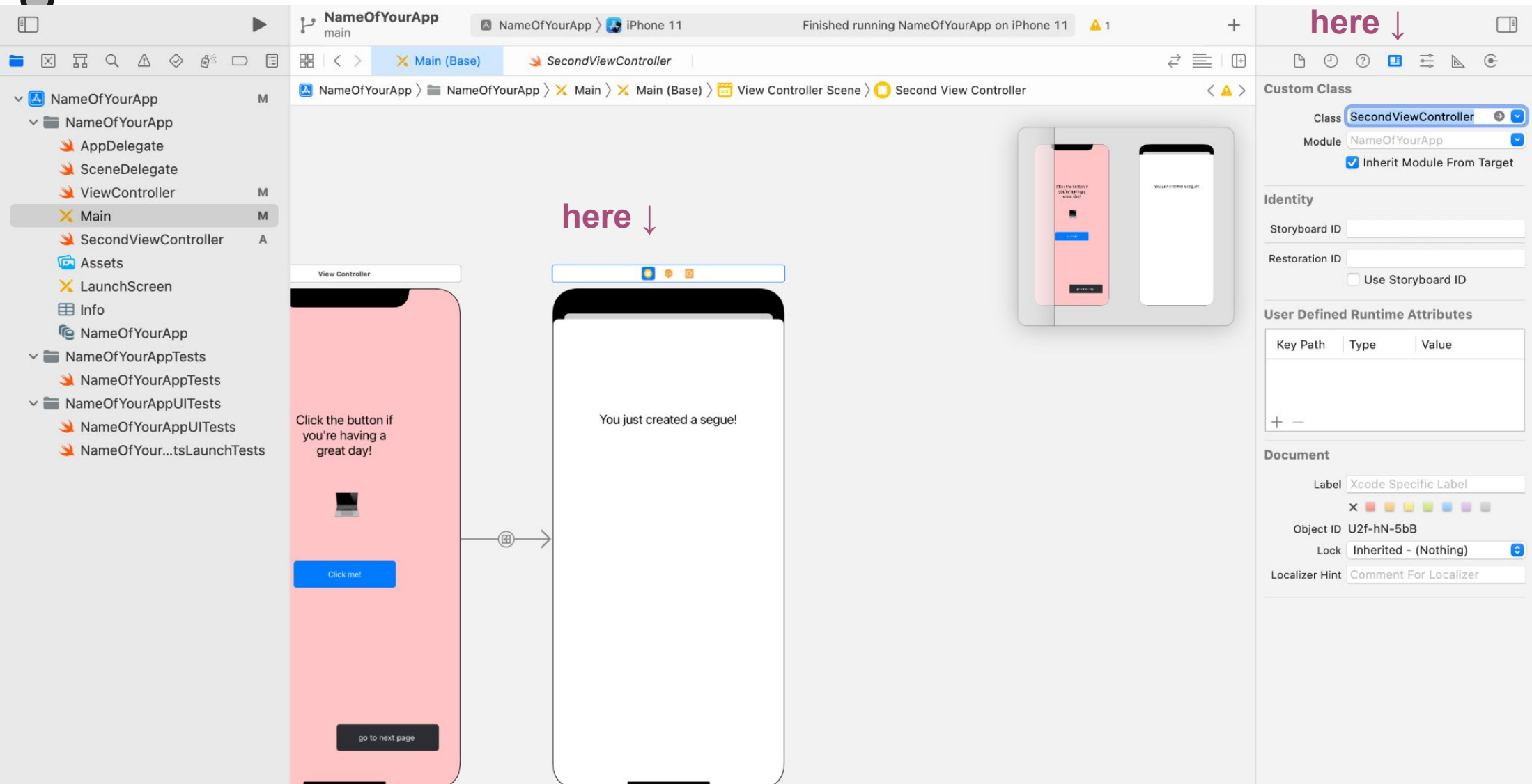
Class:

Subclass of: 

☐ Also create XIB file

Language: 

Give your new file a name (no spaces). Select the subclass as UIViewController (as this is what we want our code to correspond to), and be sure the language is Swift.



Lastly, in our Main workspace, we want to connect the code file to the screen we have added. We can click on the yellow circle on the bar right above our view controller, and then open the Identity Panel in the attributes inspector. Finally, input the name of your code file as the Class.



The screenshot displays the Xcode development environment for an iOS application named "NameOfYourApp". The main workspace shows a storyboard with two view controllers. The first view controller (Main) has a pink background, a text label "Click the button if you're having a great day!", a blue button labeled "Click me!", and a dark button labeled "go to next page". The second view controller (SecondView) has a white background and a text label "You just created a segue!". A segue arrow connects the two view controllers. The right-hand side of the interface features three panels: "Custom Class" (showing "SecondViewController" as the class for the selected view controller), "Identity" (showing "Storyboard ID" and "Restoration ID" fields), and "User Defined Runtime Attributes" (a table for defining attributes). The "Document" panel at the bottom shows the "Xcode Specific Label" and "Object ID" (U2f-hN-5bB).

Custom Class

Class: **SecondViewController**

Module: **NameOfYourApp**

☒ Inherit Module From Target

Identity

Storyboard ID:

Restoration ID:

☐ Use Storyboard ID

User Defined Runtime Attributes

Key Path	Type	Value
+ -		

Document

Label: **Xcode Specific Label**

Object ID: **U2f-hN-5bB**

Lock: **Inherited - (Nothing)**

Localizer Hint: **Comment For Localizer**

You're all set with Xcode basic foundations!





End of Project Demonstration Instructions

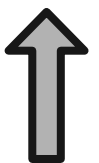




Beyond today's seminar.

What are the possibilities?

- Implementation of Swift packages & databases
 - SwiftyJSON for utilizing APIs
 - **Firestore** (a Google Cloud service with databases, authentication, ...)
 - Hundreds (or thousands) more packages are available!
- Interested in **machine learning** models?
 - Check out Apple's Core ML. Create ML app also allows you to create and train Core ML models without complex code!
- Interested in **gaming and AR**?
 - Check out ARKit from Apple!





Additional Resources

Youtube: iOS Academy

<https://www.youtube.com/@iOSAcademy>

Apple's Course: iOS App Dev Tutorials 🏆

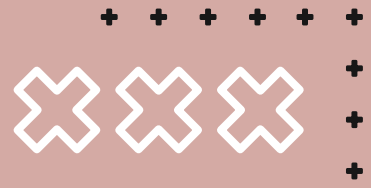
<https://developer.apple.com/tutorials/app-dev-training>

Youtube: More Tutorials & Projects

<https://www.youtube.com/@CodeWithChris/videos>

Want to become an iOS developer?

Register for the Apple Developer Program. Be able to test your apps directly on your devices, launch for beta-testing, and to the app store!





Thank

Do you have any questions?

you!

@kamrynohly

